OpenGL三维图形编程

纹理映射（下）

纹理高级技术

- 一维纹理
- 纹理代理
- 驻留纹理
- 自动纹理坐标
- 环境映射
- 纹理矩阵
- 齐次坐标
- 细节多层次（mipmap）

一维纹理

- 与二维纹理相关函数类似，target参数使用GL_TEXTURE_1D
  - 从内存纹理数据定义纹理
  - 从帧缓存数据创建纹理
  - 使用内存数据替换纹理图像
  - 使用帧缓存数据替换纹理图像
  - ……

纹理代理

- 纹理的约束较多
  - 宽高有限制、内部格式多变……
- 纹理代理
  - 一种查询机制，精确的确定当前OpenGL是否支持某种指定的纹理

纹理代理

- 复用***glTexImage2D()***函数
  - *void* ***glTexImage2D****(GLenum target, GLint level, GLint internalFormat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)*
    - *target* — 设置为*GL_PROXY_TEXTURE_2D*
    - *pixels* — 设置为NULL
    - 其余参数设置为将要创建的纹理的相应值

纹理代理

- 查询纹理状态
  - *void* ***glGetTexLevelParameter****{if}****v****(GLenum target, GLint level, GLenum pname, TYPE *params)*
  - 如果没有足够的资源创建所需纹理，则对应状态变量的查询结果为0

纹理代理

- 程序示例
  - texprox\Debug\texprox.exe
  - 查询内部格式是否为GL_RGBA8

```
glTexImage2D(GL_PROXY_TEXTURE_2D, 0,
  GL_RGBA8, 64, 64, 0, GL_RGBA,
  GL_UNSIGNED_BYTE, NULL);
glGetTexLevelParameteriv(GL_PROXY_TEXTUR
  E_2D, 0, GL_TEXTURE_INTERNAL_FORMAT,
  &format);
```

驻留纹理

- 纹理存取机制
  - 三级：显存（video memory），内存（ram），硬盘（file）
  - 一般情况下，纹理以文件形式存在，在硬盘上
  - 用户自己将纹理文件从硬盘调入内存
  - 创建纹理对象时（即使用***glTexImage2D***函数和第一次使用***glBindTexture***函数时），纹理数据从内存调入显存
  - 开始使用纹理时（即第二次调用***glBindTexture***函数时），纹理数据已经在显存中；如果纹理不在显存中，则系统会隐式的将纹理调入显存。总之，对纹理的处理是在显存中进行的

驻留纹理

- OpenGL为应用程序建立起了高性能的纹理工作集，称为“驻留（resident）”
  - 实际上就是：保持纹理对象及其数据在显存中，而不是在内存中

驻留纹理

- 显存有限，因此并不是所有纹理都可以驻留
  - 查询单个纹理的驻留情况
    - 首先绑定该纹理（激活），然后以参数GL_TEXTURE_RESIDENT调用函数glGetTexParameter*v()查询
  - 查询多个纹理的驻留情况
    - *GLboolean* ***glAreTexturesResident****(GLsizei n, const GLuint*textureNames, GLboolean *residences)*
    - 纹理全部驻留，返回*GL_TRUE，residences不变*
    - 纹理部分驻留，返回*GL_FALSE，residences相应改变*

驻留纹理

- 驻留策略
  - 为纹理对象指定驻留优先级
    - *void* ***glPrioritizeTextures****(GLsizei n, const GLuint *textureNames, const GLclampf *priorities)*
    - 优先级范围在[0.0，1.0]之间，0最低，1最高
  - 为当前绑定纹理指定优先级
    - 以GL_TEXTURE_PRIORITY为参数调用glTexParameter*()
  - OpenGL对相同优先级的纹理对象采用“最近最少使用（LRU）”策略

驻留纹理

- 有效使用驻留纹理的一个方法
  - 分析场景，估计场景要使用的纹理的大小；使用***glCopyTexImage2D***函数创建若干“空”纹理对象，直到显存全部被占满（***glAreTexturesResident***返回GL_FALSE）
  - 往内存中调入纹理数据，同时，采用自己的纹理调度策略，将硬盘中的纹理文件读入内存。譬如，单独开启一条线程实时监测
  - 采用***glTexSubImage2D***方法替换纹理内容

驻留纹理

- 上述方法的效率体现在
  - 将费时操作全部安排在初始化时进行，主要是创建所有纹理对象
  - 尽量使用高效的函数
    - ***glCopyTexImage2D***比***glTexImage2D***效率高
    - ***glTexSubImage2D***比重新创建纹理对象效率高
  - 用户自定义的驻留机制一般要比OpenGL缺省的有效

自动纹理坐标

- 预备知识
  - 三维投影空间中平面的坐标表示
    - π（a，b，c，d）
  - 三维投影空间中点的坐标表示
    - X（x，y，z，w）
  - 点到平面的距离
    - d＝π•X＝ax＋by＋cz＋dw
    - 当点X在平面π上时，d＝0，即π•X＝0

自动纹理坐标

- 物体上各点X到平面π的距离
  - 如果平面π是相对于物体定义的，即π定义在物体的坐标系中，那么，物体移动时，π也移动，物体上各点相对于π的距离不变
  - 如果平面π是相对于人眼定义的，即π定义在人眼坐标系中，那么物体移动而人眼不动时，π不动，物体上各点相对于π的距离发生改变

自动纹理坐标

- 自动纹理坐标就是将X的纹理坐标定义为物体点X到平面π的距离
  - $d = \pi \cdot X = ax + by + cz + dw$
- π有两种指定方式
  - 相对于物体定义π的位置；物体移动时，纹理坐标不变，纹理跟随物体一起移动
  - 相对于人眼定义π的位置；物体移动时，纹理坐标变动，纹理相对于物体变动，相对于人眼不动

自动纹理坐标

- 纹理坐标生成函数
  - *void* ***glTexGen****{ifd}(GLenum coord, GLenum pname, TYPE param)*
    - *coord* —指定哪一个纹理坐标需要自动生成，*GL_S, GL_T, GL_R,* 或 *GL_Q*
    - *pname* — 必须为*GL_TEXTURE_GEN_MODE*
    - *param* — 为*GL_OBJECT_LINEAR* 或 *GL_EYE_LINEAR*，纹理坐标的生成方式

自动纹理坐标

- 参考平面 π 的指定
  - *void* ***glTexGen****{ifd}****v****(GLenum coord, GLenum pname, TYPE *param)*
    - *coord* — 指定哪一个纹理坐标需要自动生成，*GL_S, GL_T, GL_R,* 或 *GL_Q*
    - *pname* — 为*GL_OBJECT_PLANE 或 GL_EYE_PLANE*
    - *param* — 定义 π 的四元数组（a，b，c，d）

自动纹理坐标

- 程序示例：texgen\Debug\texgen.exe

环境映射

- 其目的是为了渲染完全反射的物体，譬如，玻璃球，银球
  - 首先获取银球对外界反射的图像，可用广角镜或语言摄像机近似获得
  - 然后把该图像当做纹理图进行映射

环境映射

纹理矩阵

- 每一个顶点都指定了两种坐标：空间坐标，纹理坐标
- 坐标变换实际上就是矩阵变换
  - 坐标变换的常见方式：平移，旋转，缩放
  - 每一种操作对应一个矩阵变换

纹理矩阵

- 空间坐标可以通过一系列操作来改变

```
glMatrixMode(GL_MODELVIEW);
glRotated(...);
```

- 纹理坐标也可以通过相同的操作来改变

```
glMatrixMode(GL_TEXTURE);
glRotated(...);
```

齐次坐标

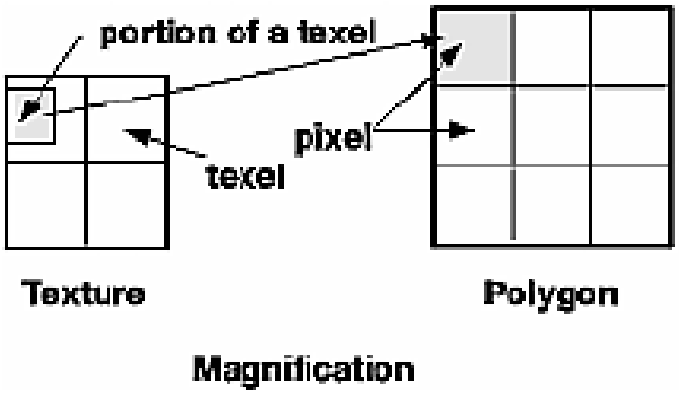
- 齐次坐标（homogeneous coordinates）
  - 是对普通坐标的扩展
  - 二维平面点（x，y）的齐次坐标表示为（x，y，w）
    - （x，y，w）→（x / w，y / w）
  - 三维空间点（x，y，z）的齐次坐标表示为（x，y，z，w）
    - （x，y，z，w）→（x / w，y / w，z / w）

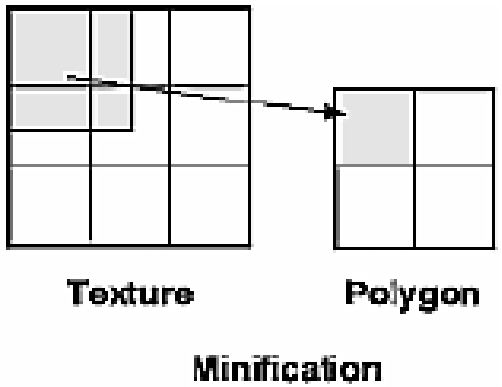
齐次坐标

- 两个重要性质
  - 对于非0的缩放因子k，齐次坐标表示同一点
    - （x，y，z，w）＝k（x，y，z，w）
  - 当w为0时，齐次坐标表示无穷远点
    - （x，y，z，w）→（x／w，y／w，z／w）

细节多层次（mipmap）

- 回顾纹理过滤
  - 纹理图单元（纹元，texel）和屏幕像素单元大小往往不是一对一的匹配

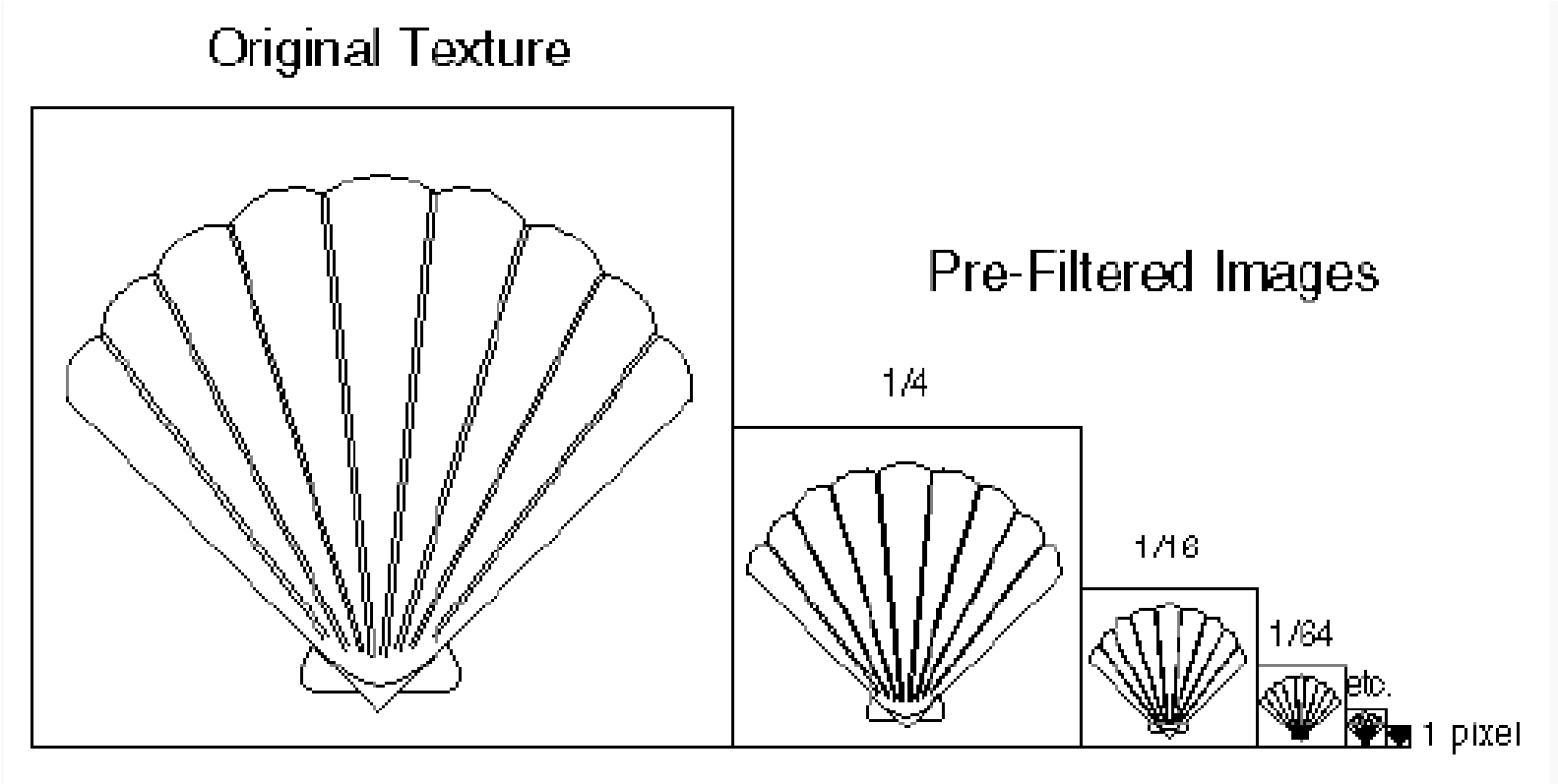




细节多层次（mipmap）

- mipmap实际上就是预过滤，预先生成一批由大到小的纹理图，根据物体所占像素的大小，选择对应的纹理进行映射
- 又称为LoD（Level of Detail）技术

细节多层次（mipmap）



细节多层次（mipmap）

- 必须为每一分辨率的纹理调用一次***glTexImage2D***函数，并设置*level*参数相应值，0为最高分辨率
  - *void **glTexImage2D**(GLenum target, GLint level, GLint internalFormat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)*

细节多层次（mipmap）

- 使用mipmap的几个要点
  - 由大到小所有的纹理，其宽、高必须都满足2的幂次
  - 从最大尺寸到最小尺寸（1×1）的所有纹理都必须给出，不得遗漏，每一层次的宽高都相差2倍
  - 各个尺寸的纹理都可以单独指定，相互之间可以完全不相关
  - mipmap如何与物体关联，由OpenGL负责

细节多层次（mipmap）

- 程序示例：mipmap\Debug\mipmap.exe

细节多层次（mipmap）

- 自动mipmap的生成
  - *int* ***gluBuild2DMipmaps****(GLenum target, GLint components, GLint width, GLint height, GLenum format, GLenum type, void *data)*
    - 参数意义与***glTexImage2D***完全相同
    - 如果*width, height*不是2的幂次，则***gluBuild2DMipmaps***负责缩放

细节多层次（mipmap）

- 高级mipmap
  - OpenGL1.2及后续版本引进了若干其它mipmap特性
  - 课程内不予讨论，可参见相关文档，最权威的是OpenGL specification

mipmap下的过滤处理

- 纹理放大（GL_TEXTURE_MAG_FILTER）
  - 仅仅使用level为0的纹理

mipmap下的过滤处理

- 纹理缩小（ GL_TEXTURE_MIN_FILTER ）
  - 指定过滤方式为GL_NEAREST或GL_LINEAR时，也仅仅使用level为0的纹理
  - 另有四种过滤方式，在mipmap层次中寻找合适的纹元
    - GL_NEAREST_MIPMAP_NEAREST
    - GL_LINEAR_MIPMAP_NEAREST
    - GL_NEAREST_MIPMAP_LINEAR
    - GL_LINEAR_MIPMAP_LINEAR

mipmap下的过滤处理

- GL_NEAREST_MIPMAP_NEAREST
  - 寻找最合适的一层mipmap，再应用GL_NEAREST规则寻找合适的纹元
- GL_LINEAR_MIPMAP_NEAREST
  - 寻找最合适的一层mipmap，再应用GL_LINEAR规则进行加权平均
- GL_NEAREST_MIPMAP_LINEAR
  - 寻找两层对应的mipmap，在每一层中应用GL_NEAREST规则寻找合适的纹元，再对所得的两个值进行加权平均
- GL_LINEAR_MIPMAP_LINEAR
  - 寻找两层对应的mipmap，在每一层中应用GL_LINEAR规则进行加权平均，再对所得的两个值进行加权平均

mipmap下的过滤处理

- GL_NEAREST，点采样
- GL_LINEAR，双线性采样
- GL_LINEAR_MIPMAP_LINEAR，三线性采样

实际应用中的LoD

- LoD最重要的是思想
  - 实际应用时，常常采用自己实现的LoD技术，而不使用OpenGL所缺省支持的

实际应用中的LoD

- 一个自定义LoD的例子
  - 定义各级纹理
  - 管理内存，实施调度算法
  - 各级不同层次的纹理之间的拼接